# 3D printer improvements

From the community for the community, mostly via heavily customized tools

Some links about me. Many of my 3D designs are free. I also post on Google+ and in another blog, oz4.us

## Sticky posts

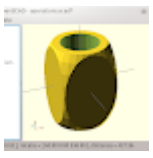
survival kit, openscad tutorial, material for artists, what cannot be 3D printed, oz4.us...

Tweets by tridimake

## Popular posts

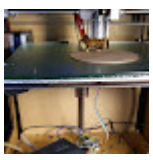
Replacing LinuxCNC with a Smoothieboard for CNC milling. Not as easy as for 3D printing!
I was running LinuxCNC for the last years and I wanted a smaller setup, with room for improvement. Since I had a Smoothieboard around, whi...

How to use Openscad (1), tricks and tips to design a parametric 3D object
Part 1/5: Introduction to constructive solid geometry with Openscad Some technical and non-technical people keep asking me how I create...



How to use Openscad (2): variables and modules for parametric designs
Part 2/5: Variable and parametric design The previous part addressed the basics of Openscad. It relied mostly on "immediate values...

How to use Openscad (4): children and advanced topics
Part 4/5: children, factorized placement and chained hulls Yes, it was made with Openscad and it is parametric! (extreme collaborativ...



Cooling 3D prints with an aquarium pump
3D printing cooled with an aquarium pump! It works well and it cannot be quieter. T he pump is bulky, but the overall is a very compact...

review (32) general (26) issue (16) mechanical (16) replacement (14) software (13) addon (12) bowden tube (10) business (10) consumables (10) heat (10) feeder (8) hot end (8) weird (7) motor (6) off-topic (6) trick (6) electric (4) bed (3) cnc (2)

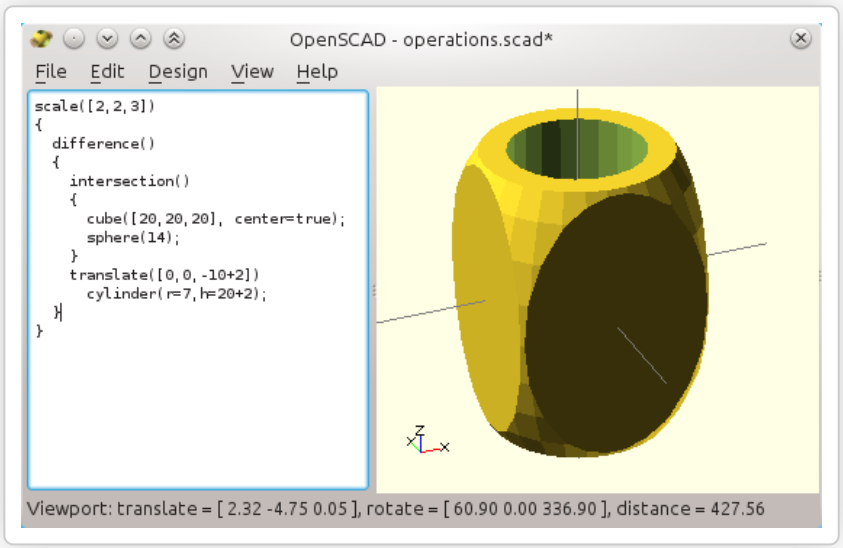
Thursday, September 11, 2014

## How to use Openscad (2): variables and modules for parametric designs

### Part 2/5: Variable and parametric design

The previous part addressed the basics of Openscad. It relied mostly on "immediate values": we were providing dimensions as explicit numbers. If you want to tweak the design dimensions, then you need to parse the scad source code and fix the numbers all everywhere.

In fact "hard coded" numbers must be avoided. As opposed to "static" designs, *parametric designs* give the flexibility to tune the numbers very efficiently at one place only. The good point is that Openscad is one of the best tool to do so.




A larger mug by using the scale operator (from the "basic" tutorial).
This is still *not parametric*, as numbers are hard-coded (a bad practice).

See how the numbers in our mug design above depends on each other? If the cube is to be made taller for any reason, then the intersecting sphere and hollow cylinder must be tuned accordingly in the source code. And this is *exactly* something a computer can do better than us.

So let us first convert this design to a parametric version, i.e. a design that can be tweaked with a small set of parameters that all have a clear role (width, height and so).

This article is part of a longer serie:

1. Introduction to constructive solid geometry with OpenSCAD
2. **Variables and modules for parametric designs**
3. Iteration, extrusion and useful parametrized CSG techniques
4. Children, factorized placement and chained hulls

We are first going to replace most of the numbers by straightforward expressions that refer to **variables** instead of numbers.

A variable is just an "alphanumeric tag": it is a name made of letters that mean something for you. You can also use underscores and digits, but the first letter must be a letter and no space is allowed in the name: hence "width", "my_cube_width", "cubeDim1" are three valid variable names (without the double quotes!).

Once a variable is given a value, it can be used *as if it was a number*. There are two advantages to use variables: readability and convenience.

```
body_diameter=20;
```

It is easier to read a source code with variables because you can give meaningful names to them: this "body_diameter" makes more sense than any number buried in a mathematical expression.

For example, you can now replace an obscure shape like this:

```
cylinder(r=10);
```

By this one, which is much more readable:

```
body_diameter=20;
cylinder(r=body_diameter/2)
```

In fact, readability is a second benefit, because the foremost advantage is to get rid of rogue "20" or "10" in the source code that are anonymously related to the body size!

## Parametric rewrite of the mug source (i.e. *depends on variables*)

Back to our mug to make it parametric. First, extract and name appropriately the main characteristics of your design:

```
width=40;
height=60;
bottom_thickness=2;
wall_thickness=5;
```

Then rewrite the former anonymous numbers, by using these variables and mathematical expressions:

```
difference()
{
  intersection()
  {
    cube([width,width,height], center=true);
    scale([1,1,height/width]) // so that Z=1 is now...
      sphere(width/2 * sqrt(2)); // close to the cuboid edges
  }
  translate([0,0,-height/2+bottom_thickness])
    cylinder(r=width/2-wall_thickness,h=height+0.1);
}
```

A "good" Openscad source code should almost have no numerical values. Most of the internal stuff is computed out of a minimum set of variables that make sense in the design. Especially, there should be one and only one place to set the geometric dimensions.

**A side note on scale and rotate.**

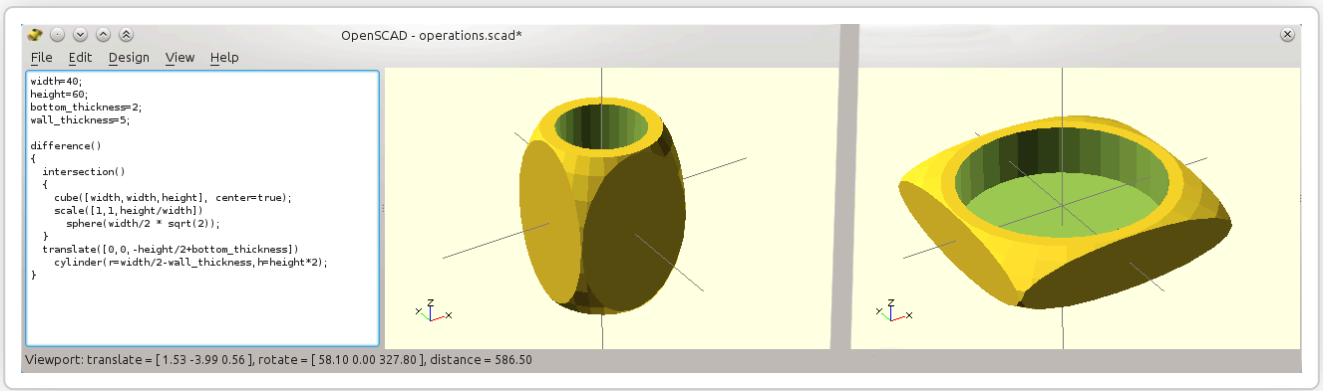
In the above parametrized source, the formerly almost global "scale( )" operator was moved "down in the CSG tree". Just like rotation, scale is an annoying operator because all subsequent axis geometry gets modified. Nothing is "square" anymore (or respectively, well-oriented)! A good indicator that an operator is used "too soon" in the CSG tree is when you suddenly feel the need to reverse its effect (un-scale or de-rotate before you add sub-shapes).

Better use "scale( )" sparingly, and only when required, closest to the shape that needs it. Here we apply it only to one primitive, the sphere, to elongate it vertically so it matches the cuboid. And this is exactly what we wanted: it makes life easier than to intersect a regular cube with a sphere and then elongate the whole even though the result is the same. Why? Because if you needed to add a regular cylindrical hole in the intersection for example, it would be itself ellipsoidal!

*Note:* so far, better consider Openscad variables as *constants*! With Opensacd, a variable cannot be re-defined once set (usually at the top of the file or at the top of a module)! More on this later.

## An immediate effect: multiple shapes with one source code!

As shown below, the parametrized source code let us generate a mug or an ashtray just by changing the *width* and *height* variables to 40 and 60 mm, or 80 and 20 mm respectively (well, a plastic ashtray is not a good idea).




Two "mugs" made from the same source code: only two numbers were changed (*height* and *width*) ! Parametric designs are probably one of the most important characteristics of Openscad.

Often, we have a shape to repeat in different places, like PCB support pillars with the hole for a screw and a bevel at the base. But how do we combine shapes conveniently?

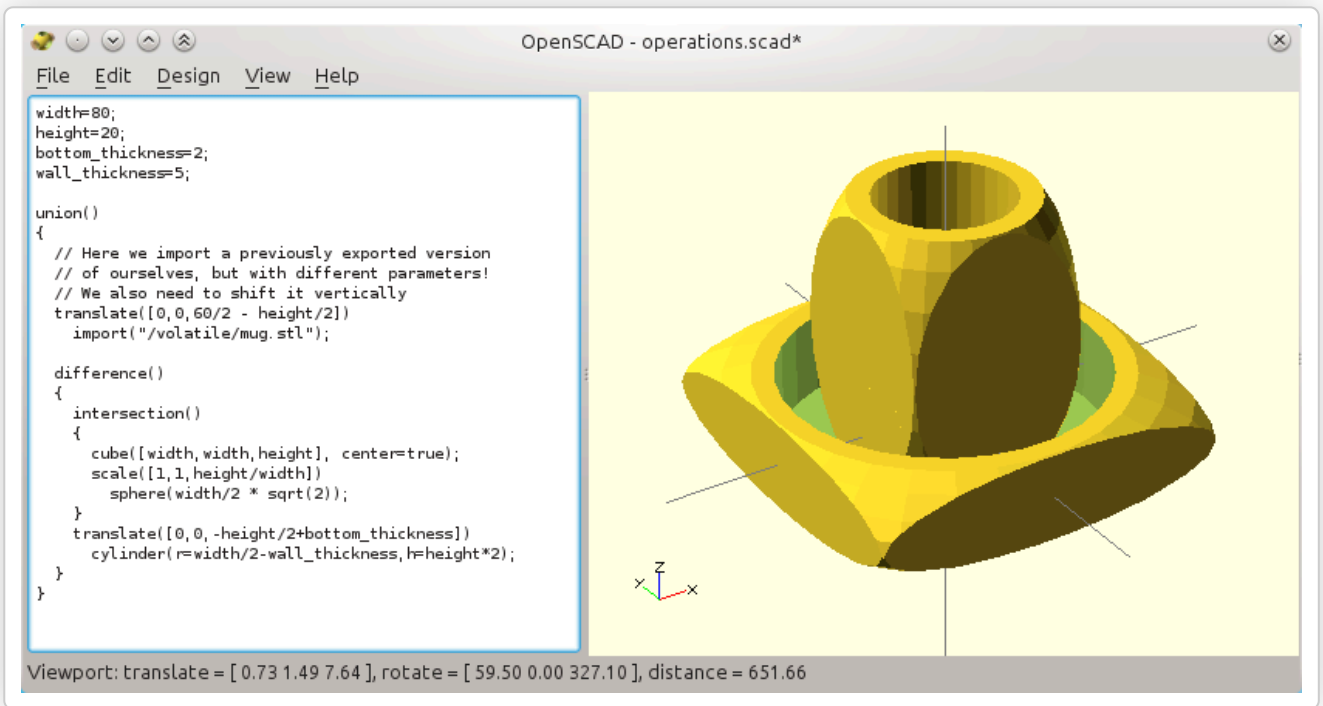
Imagine a weird case: to reduced spilled coffee, we are going to design a mug with a cup attached to it, using the above two shapes.

## Export/import STL files

Openscad knows how to **import** raw STL files (triangular meshes, that we described in the first part of this serie). This is mostly useful to tweak an existing design for which no source is given, or not compatible with Openscad. For example, I sometimes design and add manual support walls this way.

So we could first generate the mug, then export it to an STL file, and finally import the mug STL in our modified source code for the cup.

Here is how it would look like here:




A nasty way to import a parametric copy of ourselves, with the "import" function of Openscad and after the mug shape is exported to an STL file. But this is not the right way to do, better use a *module* of course!

It works as you can see, but this is an ugly solution. See also the rogue translation in the source code to fix the position of the "import( )" statement (the value depends on the height of the internal mug, no more available, and the height of the current mug shape). Do not do that!

Now, so shall we copy/paste the mug code to get two versions-in-one? Obviously not: once again, the less the number of lines of source code, the less bugs. First, copying/pasting source code is very wrong because any bug left in the copied source will be *duplicated.* Second, any subsequent improvement need to be re-inserted in the many copies. Third, the copies tend to diverge from each other with time while the unique and "canonical" shape would better have added options instead for more re-usability in the first place.

## Writing a module: recyclable and parametric shapes

Instead of copy/pasting source code, Openscad offers **modules** that are the right choice here. Modules are like functions (or classes) for the readers who practiced a bit of programming.

The idea of a module is to create a "generic" shape which depends on some parameters. In fact a module creates a new shape exactly like we have primitive shapes, except that *we* define the shape!

Here is how we have to modify our source code:

```
module mug(width, height, bottom_thickness=2, wall_thickness=5)
{
  difference()
  {
    intersection()
    {
```

```
        cube([width,width,height], center=true);
        scale([1,1,height/width])
          sphere(width/2 * sqrt(2));
      }
      translate([0,0,-height/2+bottom_thickness])
        cylinder(r=width/2-wall_thickness,h=height+0.1);
    }
  }
```

This module generates a new *mug shape*, just like "cube" or "cylinder" create their respective shapes. Our mug is certainly no more a *primitive* one though, and it requires up to four parameters, that we had previously introduced with the notion of *variables*.

Here, two of the parameters have *default values*: they are not compulsory when a new mug is created, even though you can *override* the default value by setting their value. This is quite convenient because we probably do not care much about the thickness except is rare cases.

**No shape is defined, blank (or un-refreshed display) in Openscad!**

Now if you try and run the above, you get nothing new nor updated. In fact the Openscad console tells this:
`ERROR: CSG generation failed! (no top level object found)`
This is one more reminder that you should better always leave the console open, or you will not always understand why nothing appears or where something is broken.

The error means "nothing is left to be drawn at the end of the code". And this is true since we just have *defined* the mug shape, but we created none so far. This error also happens in case an abusive intersection leaves no positive shape to display for example.

**Calling or creating a shape with a module**

So to *create* a version of our original mug, we have to call the module, with the following line.
The line must be placed *outside* of the module (aka shape definition), for example at the end of the file:

```
mug(width=40, height=60);
```

For the cup/ashtray, it would just be another line:

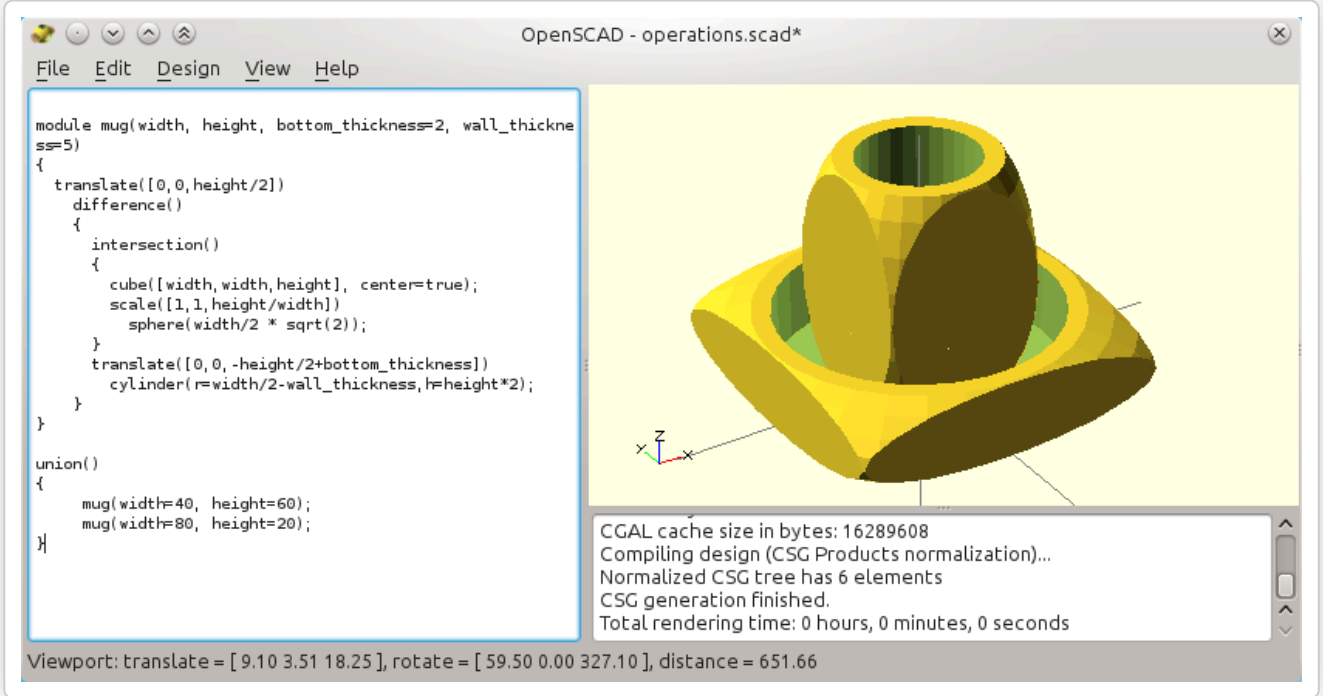
```
mug(width=80, height=20);
```

Then, our combined mug+cup shape is just the (default) union of the two shapes.

See how powerful Openscad is becoming? And there is more to come...

By the way, you will see that I also corrected the vertical offset of the mug shape within the module, so that Z=0 corresponds to the bottom of the shape once created. Indeed, nobody wants the mug "origin" to be positioned at its middle, because it makes life harder for *callers* of the mug shape (remember the ugly "translate( )" we added when importing the STL above!).

You may also notice that there are no curly braces after the "translate" within the top of the mug module. This is fully OK because there is really only one "command" that needs to be translated after, namely, the (result of the) "difference" that follows. It is up to you to use curly braces everywhere or not. I tend to drop them on short statements, or when I want less "noise".




Our coupled mug and teacup thanks to a module that builds generic (aka parametric) mug shapes.

```
module mug(width, height, bottom_thickness=2, wall_thickness=5)
```

```
{
  translate([0,0,height/2])
    difference()
    {
      intersection()
      {
        cube([width,width,height], center=true);
        scale([1,1,height/width])
          sphere(width/2 * sqrt(2));
      }
      translate([0,0,-height/2+bottom_thickness])
        cylinder(r=width/2-wall_thickness,h=height+0.1);
    }
}

// here is the real objects that are made
union()
{
  mug(width=40, height=60); // one mug shape1
  mug(width=80, height=20); // another one, different
}
```

The last "union( )" is spurious, because it is the default operation when more than one shape are provided. But it makes thing clearer.

We can also create and use convenient "local" variables in the module. Check the source below: we added the explicit "`r_of_inside`". This variable is unseen from the outside of the module (*caller* side). And even though it is used only once at the end of the module, it helps to improve readability.

Finally, note that I moved the translation by height/2 closer to the "positive" shape. Carving the cylinder is more readable without a centering. This is the same reason we moved the "scale( )" closer to the sphere.

```
module mug(width, height, bottom_thickness=2, wall_thickness=5)
{
  r_of_inside=width/2-wall_thickness;
  difference()
  {
    translate([0,0,height/2])
      intersection()
      {
        cube([width,width,height], center=true);
        scale([1,1,height/width])
          sphere(width/2 * sqrt(2));
      }
    translate([0,0,bottom_thickness])
      cylinder(r=r_of_inside,h=height+0.1);
  }
}
```

*Important note*: there are some heavy restrictions on how you can use variables. Older versions of Openscad had problems when a module called itself for example. No big problem so far, as recursion is an advanced topic that we will tackle in the next article.

## Recycling the modules from other Openscad source files

Our mug *module* can be saved to a scad file, as usual. Actually a scad file may contain more than one module.

A nice feature is that you can *import* scad files, just as we imported STL objects previously. Then, the modules are available in the new file just as if they were default shapes.

To do so you can create a new scad file and tell it to import your former mug scad file. I have to admit that the syntax is stupid given that "include" works as a regular function, why does it need to be otherwise for this one (my guess is: a clumsy heritage from the C language)?

```
include <mug.scad >
```

Now there is a special comment to do. The behavior here is just as if you had copied/pasted the scad source from "mug.scad" in place of the "include" statement. In this case, you will see the mug-with-a-cup that we defined, even without creating any shape in your new file.

So there is another, probably more useful way to recycle your old designs, thanks to another function:

```
use <mug.scad >
mug(width=40, height=60);
```

This time, the "use" statement only imports the *definition* of the modules that were defined in "mug.scad". It will not execute or create anything else though as it bypasses all immediate shapes from within mug.scad.

Again: "use" statements will never generate shapes, even though we may have left some in "mug.scad". This is *very* convenient because "mug.scad" can do something on its own without polluting the new code here (e.g. just to show a standalone working example, as we did).

*Note*: there are a lot of almost-official modules, in a library named MCAD. It brings a huge quantity of additional shapes and standard nuts, bolts and other industrial equipment. It is free as much as Openscad, and the source code is here.

## Convex hull: creating complex shapes with basic ones.

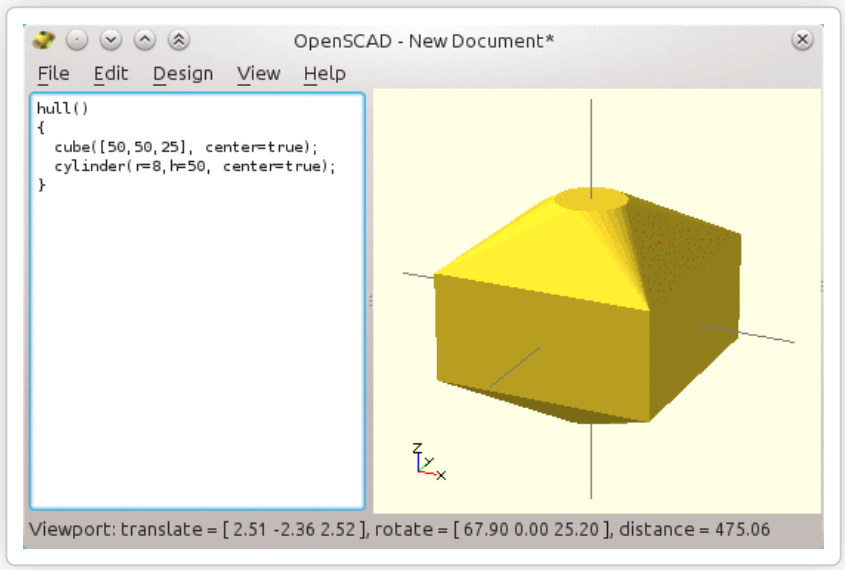
Now, life with Openscad would not be possible without another operator: the *convex hull*.

Think of it as the shape that you get when you wrap a plastic film around a bunch of shapes and heat it, with no resulting concave parts. The effect would be the same with soap (only incredibly harder to achieve in real life...)

A sample will make this clear:

```
hull()
{
  cube([10,10,10], center=true);
  translate([20,0,0]) sphere(r=3);
}
```

Below you get an animation that changes from the regular "union( )" to a "hull( )".




Animation showing the effect of "hull( )", aka "convex hull".

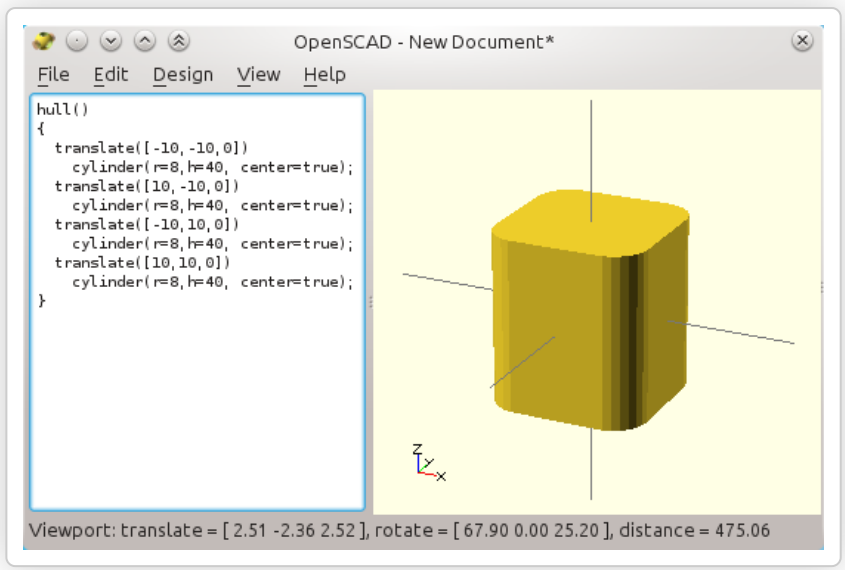
Usually no "complex" shape are used within a hull. But reciprocally, it is very rare that a complex shapes has no hulls.

There can be any number of shapes within the hull, as we do below: to build a cube with round corners, we are constructing the convex hull of four cylinders.

```
hull()
{
  translate([-10,-10,0])
    cylinder(r=8,h=50, center=true);
  translate([10,-10,0])
    cylinder(r=8,h=50, center=true);
  translate([-10,10,0])
    cylinder(r=8,h=50, center=true);
  translate([10,10,0])
    cylinder(r=8,h=50, center=true);
}
```

*Note:* there are many way to build such a shape, and using a hull is mostly overkill: a union is enough, as we will see later. Anyhow, this is just an illustration of the use of hulls, and remember how wrong it is to use immediate values when they depend on each other! All this is fixed later.

Also, remember that you can prefix a shape with % or # when you want to check its individual contribution to the union!




One way to build a (partially) rounded cube.
We will do this better later...

In fact, there are two improvements to be made, at least.

First, a hull is a lengthy operation, as Openscad needs to compute a lot of shapes. It is usually kept for shapes that cannot be achieved otherwise.

Indeed, the above is faster when we realize it no more than a union of 4 cylinders and 2 cuboids, this way:

```
union()
{
  translate([-10,-10,0]) cylinder(r=8,h=50, center=true);
  translate([10,-10,0])  cylinder(r=8,h=50, center=true);
  translate([-10,10,0])  cylinder(r=8,h=50, center=true);
  translate([10,10,0])   cylinder(r=8,h=50, center=true);
  cube([20,20+2*8,50], center=true); // space left inside
  cube([20+2*8,20,50], center=true); // and in the other way
}
```

Then, a second improvement should be made as this is no proper way to code anyway.

Remember what I said about copy/pasted code: avoid copy/pasting at all costs!

Here, it ought to be made with *loops*, i.e. repeated shapes (and not repeated typing). Which is the subject of the third article, that tackles more advanced CSG techniques in Openscad.

Posted by Jeremie Francois at 11:51 AM

Labels: review, software

## No comments:

## Post a Comment

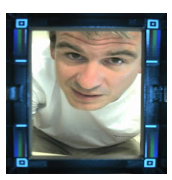
To leave a comment, click the button below to sign in with Google.

SIGN IN WITH GOOGLE

Newer Post Home Older Post

Subscribe to: Post Comments (Atom)

## About Me



**Jeremie Francois**

View my complete profile

## Follow by RSS

Posts

Comments

## Blog Archive


- ► 2017 (3)
- ► 2016 (7)
- ► 2015 (9)
- ▼ 2014 (18)
  - ► Nov 2014 (3)
  - ▼ Sep 2014 (2)
    - How to use Openscad (2): variables and modules for...
    - How to use Openscad (1), tricks and tips to design...
  - ► Aug 2014 (2)
  - ► Jul 2014 (1)
  - ► Jun 2014 (2)
  - ► May 2014 (2)
  - ► Mar 2014 (2)
  - ► Jan 2014 (4)
- ► 2013 (14)
- ► 2012 (29)


More links? my curriculum, designed 3D objects, panoramas, pictures, company, and emaiil

This work is licensed CC BY-SA 4.0. Awesome Inc. theme. Powered by Blogger.